

Name: Breno Lopes Mafra

Student ID: 101485572

Lab Exercise 3 – Screenshots

The screenshot displays the Postman interface for a REST client. At the top, the collection is named 'New Collection / Create Employee'. The request method is set to 'POST' and the URL is 'http://localhost:3000/employee'. The 'Send' button is visible. Below the request bar, the 'Body' tab is selected, showing a JSON payload in raw format. The payload is a JSON object with the following fields: 'firstname' (Pritessh), 'lastname' (Patel), 'email' (p@pp.com), 'gender' (Male), 'city' (Toronto), 'designation' (Senior Software Engineer), and 'salary' (1000). The response section at the bottom shows a status of '201 Created' with a response time of 90 ms and a body size of 509 B. The response body is a JSON object containing the same employee data plus additional fields: '_id' (6982949ed61692bbce208309), 'updatedat' (2026-02-04T00:36:46.258Z), 'created' (2026-02-04T00:36:46.258Z), and '__v' (0).

HTTP New Collection / Create Employee Save Share

POST http://localhost:3000/employee Send

Docs Params Auth Headers (9) Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "firstname": "Pritessh",
3   "lastname": "Patel",
4   "email": "p@pp.com",
5   "gender": "Male",
6   "city": "Toronto",
7   "designation": "Senior Software Engineer",
8   "salary": 1000
9 }
10
```

Body 201 Created • 90 ms • 509 B • Save Response

{ JSON Preview Visualize

```
1 {
2   "firstname": "pritessh",
3   "lastname": "patel",
4   "email": "p@pp.com",
5   "gender": "Male",
6   "city": "toronto",
7   "designation": "senior software engineer",
8   "salary": 1000,
9   "_id": "6982949ed61692bbce208309",
10  "updatedat": "2026-02-04T00:36:46.258Z",
11  "created": "2026-02-04T00:36:46.258Z",
12  "__v": 0
}
```

HTTP

New Collection / Get Employee

Save

Share

GET

http://localhost:3000/employee

Send

Docs

Params

Auth

Headers (7)

Body

Scripts

Settings

Cookies

none

This request does not have a body

Body

200 OK

92 ms

235 B

Save Response

{ JSON

Preview

Visualize

1

[]

ORGANIZATION
Breno Lopes's Org - 2...

PROJECT
Full Stack

BL

Data Explorer

My Queries

Data Modeling

CLUSTERS (1)

Search clusters

Cluster0

- admin
- db_comp3133_employee
- local

employees

Cluster0 > db_comp3133_employee > employees

View monitoring

Visualize Your Data

Documents 2

Aggregations

Schema

Indexes 2

Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain

Reset

Find

Options

25

1 - 2 of 2

```
_id: ObjectId('69828cf3b9909d03bec4e37d')
firstname: "pritesh"
lastname: "patel"
email: "p@p.com"
gender: "Male"
city: "toronto"
designation: "senior software engineer"
salary: 1000
updatedat: 2026-02-04T00:04:03.344+00:00
created: 2026-02-04T00:04:03.344+00:00
__v: 0
```

```
_id: ObjectId('6982949ed61692bbce208309')
firstname: "pritessh"
lastname: "patel"
email: "p@pp.com"
gender: "Male"
city: "toronto"
designation: "senior software engineer"
salary: 1000
updatedat: 2026-02-04T00:36:46.258+00:00
created: 2026-02-04T00:36:46.258+00:00
__v: 0
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL ... node - w4_3133_nodejs_express_mongodb_template + v [ ] [ ] ... | [ ] [ ] X
Brenos-MacBook-Air:w4_3133_nodejs_express_mongodb_template brenolopesmafra$ npm start
    at async model.save (/Users/brenolopesmafra/Documents/study-GBC/comp3133/week4/w4_3133_nodejs_express_mongodb_template/node_modules/mongoose/lib/model.js:609:5)
    at async /Users/brenolopesmafra/Documents/study-GBC/comp3133/week4/w4_3133_nodejs_express_mongodb_template/node_modules/mongoose/lib/model.js:2714:9
    at async Promise.all (index 0)
    at async Function.create (/Users/brenolopesmafra/Documents/study-GBC/comp3133/week4/w4_3133_nodejs_express_mongodb_template/node_modules/mongoose/lib/model.js:2699:11) {
  errorLabelSet: Set(0) {},
  errorResponse: {
    index: 0,
    code: 11000,
    errmsg: 'E11000 duplicate key error collection: db_comp3133_employee.employees index: email_1 dup key: { email: "p@p.com" }',
    keyPattern: { email: 1 },
    keyValue: { email: 'p@p.com' }
  },
  index: 0,
  code: 11000,
  keyPattern: { email: 1 },
  keyValue: { email: 'p@p.com' }
}
{
  firstname: 'Pritessh',
  lastname: 'Patel',
  email: 'p@pp.com',
  gender: 'Male',
  city: 'Toronto',
  designation: 'Senior Software Engineer',
  salary: 1000
}
6982949ed61692bbce208309 has been validated (but not saved yet)
Before Save
6982949ed61692bbce208309 has been saved
```

EMPLOYEE.JS:

```
const mongoose = require('mongoose');

const EmployeeSchema = new mongoose.Schema({
  firstname: {
    type: String,
    required: [true, "First Name is required"],
    minlength: [5, "First Name must be at least 5 characters long"],
    lowercase: true,
    trim: true
  },
  lastname: {
    type: String,
    required: [true, "Last Name is required"],
    minlength: [5, "Last Name must be at least 5 characters long"],
    lowercase: true,
    trim: true
  },
  email: {
    type: String,
    required: [true, "Email is required"],
    lowercase: true,
    trim: true,
    unique: true,
    match: [/.+@.+.+/, "Please fill a valid email address"]
  },
  gender: {
    type: String,
    enum: ['Male', 'Female', 'Other'],
  },
  city: {
    type: String,
    required: [true, "City is required"],
    lowercase: true,
    trim: true
  },
},
```

```
    designation: {
      type: String,
      required: [true, "Designation is required"],
      lowercase: true,
      trim: true
    },
    salary: {
      type: Number,
      min: [1000, "Salary must be at least 1,000"],
      max: [1000000, "Salary must be at most 1,000,000"],
      validate: {
        validator: Number.isInteger,
        message: '{VALUE} is not an integer value'
      }
    },
    created: {
      type: Date
    },
    updatedat: {
      type: Date
    },
  });
```

```
//Declare Virtual Fields
```

```
//Custom Schema Methods
```

```
//1. Instance Method Declaration
```

```
//2. Static method declararion
```

```
//Writing Query Helpers
```

```
EmployeeSchema.pre('save', function () {
```

```

console.log("Before Save")

let now = Date.now()

this.updatedat = now
// Set a value for createdAt only if it is null
if (!this.created) {
  this.created = now
}
});

EmployeeSchema.pre('findOneAndUpdate', function () {
  console.log("Before findOneAndUpdate")
  let now = Date.now()
  this.set({ updatedat: now })
  console.log(this.updatedat)
});

EmployeeSchema.post('init', (doc) => {
  console.log("%s has been initialized from the db", doc._id);
});

EmployeeSchema.post('validate', (doc) => {
  console.log("%s has been validated (but not saved yet)", doc._id);
});

EmployeeSchema.post('save', (doc) => {
  console.log("%s has been saved", doc._id);
});

EmployeeSchema.post('remove', (doc) => {
  console.log("%s has been removed", doc._id);
});

const Employee = mongoose.model("Employee", EmployeeSchema);
module.exports = Employee;

```

EMPLOYEE ROUTES.JS:

```
const express = require('express');
const employeeModel = require('../models/Employee');
const app = express();

//Read ALL
//http://localhost:8081/employees
app.get('/employees', async (req, res) => {
  const employees = await employeeModel.find({});
  //Sorting
  //use "asc", "desc", "ascending", "descending", 1, or -1
  //const employees = await employeeModel.find({}).sort({'firstname': -1});
```

```

//Select Specific Column
//const employees = await employeeModel.find({}).select("firstname lastname salary").sort({'salary' : 'desc'});

try {
  console.log(employees[0].surname)
  res.status(200).send(employees);
} catch (err) {
  res.status(500).send(err);
}
});

//Read By ID
//http://localhost:8081/employee?id=60174acfcde1ab2e78a3a9b0
app.get('/employee', async (req, res) => {
  //const employees = await employeeModel.find({_id: req.query.id});
  //const employees = await employeeModel.findById(req.query.id);
  const employees = await employeeModel.find({_id: req.query.id}).select("firstname lastname salary");

  try {
    res.send(employees);
  } catch (err) {
    res.status(500).send(err);
  }
});

//Search By First Name - PATH Parameter
//http://localhost:8081/employees/firstname/pritesh
app.get('/employees/firstname/:name', async (req, res) => {
  const name = req.params.name
  const employees = await employeeModel.find({firstname : name});

  //Using Virtual Field Name
  //console.log(employees[0].fullname)

  //Using Instance method
  //console.log(employees[0].getFullName())

```

```

//Using Static method
//const employees = await employeeModel.getEmployeeByFirstName(name)

//Using Query Helper
//const employees = await employeeModel.findOne().byFirstName(name)

try {
  if(employees.length != 0){
    res.send(employees);
  }else{
    res.send(JSON.stringify({status:false, message: "No data found"}))
  }
} catch (err) {
  res.status(500).send(err);
}
});

//Search By First Name OR Last Name
//http://localhost:8081/employees/search?firstname=pritesh&lastname=patel
app.get('/employees/search', async (req, res) => {
  //console.log(req.query)
  if(Object.keys(req.query).length != 2){
    res.send(JSON.stringify({status:false, message: "Insufficient query parameter"}))
  }else{
    const fname = req.query.firstname
    const lname = req.query.lastname
    //{ $or: [{ name: "Rambo" }, { breed: "Pugg" }, { age: 2 } ] },
    //const employees = await employeeModel.find({ $and: [{firstname : fname}, {lastname : lname}]});
    const employees = await employeeModel.find({ $or: [{firstname : fname}, {lastname : lname}]});
    ///Use below query for AND condition
    //const employees = await employeeModel.find({firstname : fname, lastname : lname});

    try {
      if(employees.length != 0){
        res.send(employees);
      }else{

```

```

    res.send(JSON.stringify({status:false, message: "No data found"}))
  }
} catch (err) {
  res.status(500).send(err);
}
}
});

```

//Search By salary > 1000

//http://localhost:8081/employees/salary?value=1000

```

app.get('/employees/salary', async (req, res) => {
  //console.log(req.query)
  if(Object.keys(req.query).length != 1){
    res.send(JSON.stringify({status:false, message: "Insufficient query parameter"}))
  }else{
    const salary = req.query.value

    //const employees = await employeeModel.find({salary : {$gte : salary}});
    const employees = await employeeModel.find({}).where("salary").gte(salary);
    // <= 10000
    //const employees = await employeeModel.find({salary : {$lte : salary }});

    try {
      if(employees.length != 0){
        res.send(employees);
      }else{
        res.send(JSON.stringify({status:false, message: "No data found"}))
      }
    } catch (err) {
      res.status(500).send(err);
    }
  }
});

```

//Some more test queries

//http://localhost:8081/employees/test

```

app.get('/employees/test', async (req, res) => {
  try {
    const employees = employeeModel.
      find({})
      .where('lastname').equals('patel')
      .where('salary').gte(1000.00).lte(10000.00)
      .where('firstname').in(['pritesh', 'moksh'])
      .limit(10)
      .sort('-salary')
      .select('firstname lastname salary')
      .exec((err, data) => {
        if (err){
          res.send(JSON.stringify({status:false, message: "No data found"}));
        }else{
          res.send(data);
        }
      });
  } catch (err) {
    res.status(500).send(err);
  }
});

```

//Create New Record

/*

//Sample Input as JSON

//application/json as Body

```

{
  "firstname": "Pritesh",
  "lastname": "Patel",
  "email": "p@p.com",
  "gender": "Male",
  "city": "Toronto",
  "designation": "Senior Software Engineer",
  "salary": 10000.50
}

```

*/

//http://localhost:8081/employee

```

app.post('/employee', async (req, res) => {
  console.log(req.body)
  try {
    const employee = await employeeModel.create(req.body);
    res.status(201).send(employee);
  } catch (err) {
    console.error("Create employee error:", err);
    if (err && err.code === 11000) {
      return res.status(409).send({ status: false, message: "Email already exists", keyValue: err.keyValue });
    }
    if (err && err.name === "ValidationError") {
      const details = Object.values(err.errors).map((e) => e.message);
      return res.status(400).send({ status: false, message: "Validation failed", details });
    }
    res.status(400).send({
      status: false,
      message: "Bad Request",
      error: err && err.message ? err.message : "Unknown error",
      name: err && err.name ? err.name : "UnknownError",
    });
  }
});

```

//Update Record

//http://localhost:8081/employee/60174acfcde1ab2e78a3a9b0

```

app.patch('/employee/:id', async (req, res) => {
  try {
    console.log(req.body)
    const employee = await employeeModel.findOneAndUpdate({ _id: req.params.id}, req.body, {new: true})
    //const employee = await employeeModel.findByIdAndUpdate(req.params.id, req.body, {new: true})
    res.send(employee)
  } catch (err) {
    res.status(500).send(err)
  }
});

```

//Delete Record by ID

```

//http://localhost:8081/employee/5d1f6c3e4b0b88fb1d257237
app.delete('/employee/:id', async (req, res) => {
  try {
    const employee = await employeeModel.findByIdAndDelete(req.params.id)

    if (!employee)
    {
      res.status(404).send(JSON.stringify({status: false, message:"No item found"}))
    }else{
      res.status(200).send(JSON.stringify({status: true, message:"Record Deleted Successfully"}))
    }
  } catch (err) {
    res.status(500).send(err)
  }
})

//Delete Record using findOneAndDelete()
//http://localhost:8081/employee/delete?emailid=5d1f6c3e4b0b88fb1d257237
app.get('/employee/delete', async (req, res) => {
  try {
    const employee = await employeeModel.findOneAndDelete({email: req.query.emailid})

    if (!employee)
    {
      res.status(404).send(JSON.stringify({status: false, message:"No item found"}))
    }else{
      //employee.remove() //Update for Mongoose v5.5.3 - remove() is now deprecated
      res.status(200).send(JSON.stringify({status: true, message:"Record Deleted Successfully"}))
    }
  } catch (err) {
    res.status(500).send(err)
  }
})

module.exports = app

//Insert Multiple Records
/*

```

```
employeeModel.create(

[{"firstname":"Keriann","lastname":"Qualtro","email":"kqualtro3@mediafire.com","gender":"Female","city":"Ulricehamn",
,"designation":"Nurse Practicioner","salary":"9288.95"},

{"firstname":"Bette","lastname":"Elston","email":"belston4@altervista.org","gender":"Female","city":"Xinhang","designa
tion":"Staff Accountant III","salary":"3086.99"},
  {"firstname":"Editha","lastname":"Feasby","email":"efeasby5@ovh.net","gender":"Female","city":"San
Francisco","designation":"Mechanical Systems Engineer","salary":"1563.63"},
  {"firstname":"Letizia","lastname":"Walrond","email":"lwalrond6@ibm.com","gender":"Male","city":"Ricardo Flores
Magon","designation":"Research Associate","salary":"6329.05"},

{"firstname":"Molly","lastname":"MacTrustrie","email":"mmactrustrie7@adobe.com","gender":"Female","city":"Banjarej
o","designation":"Quality Control Specialist","salary":"4059.61"}]
)
*/
```